Temperature Propagation Simulation Software: Project Report

Franz Etzenbach & Lina Yushuvayev

December 8, 2025

Abstract

This document will be an in-depth breakdown of a Temperature Propagation Simulator modeled via Python code, aiming to explain the computational methods used in this software, and analyze the data gathered based on varying parameters detailed throughout this report. The goal of this project was to develop a software that could predict the propagation of temperature on varying surfaces (nth-dimensional polygons on a two-dimensional plane) using the transient heat equation, allowing users to input heat sources (either static or oscillatory), and model the temperature propagation over time on a three-dimensional grid. In successfully coding and modeling this simulation, these results can be used to understand how to best model thermal distributions and set up temperature–dependent environments, such as aquariums and computers.

Here is the link for the GitHub repository to access the code:`https://github.com/franzlleonard/temperature_fish_simulation/tree/main/ts_.py`

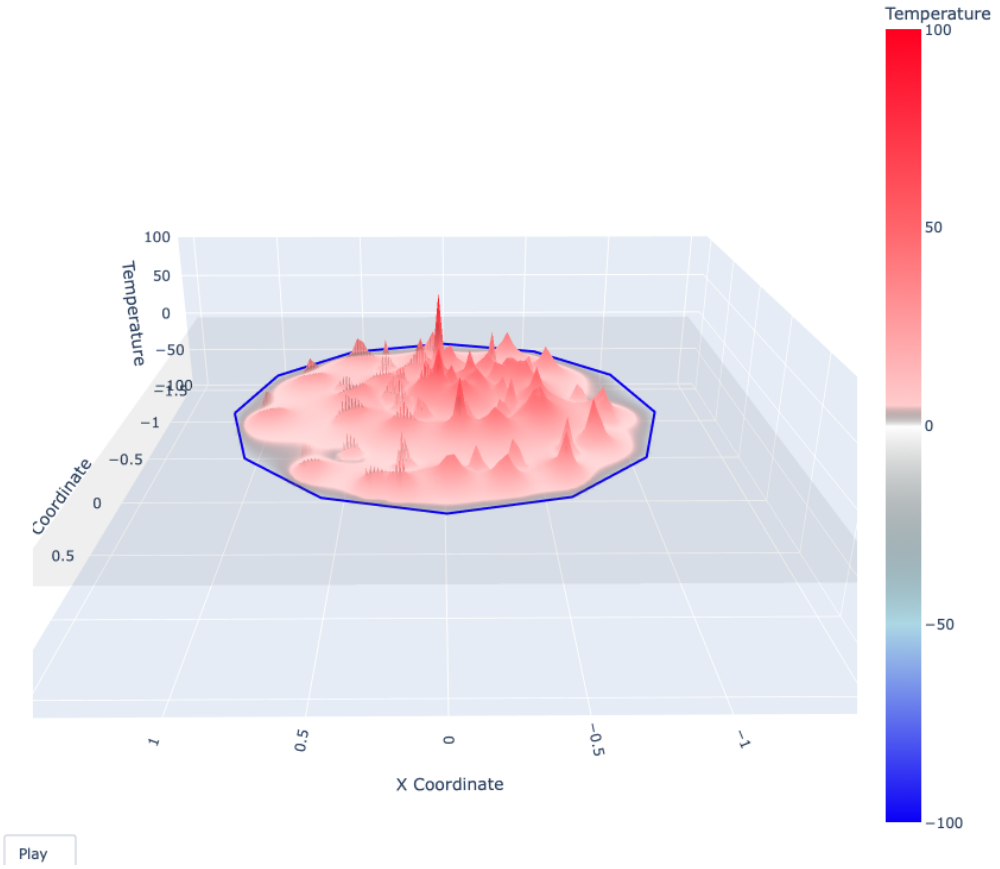

Figure 1: Example of Software Simulated Temperature Propagation

1 Introduction

1.1 Purpose

Our group decided to develop a software simulation that could model varying heat source placements (either static or oscillatory) to find the temperature at each position in the pond over time, with the goal to find an ideal temperature range that would allow for fish survival. After developing our software, we found that we could modify the core temperature simulation to other applications, such as how to best set up a computer and optimize thermal distribution (with regards to heating) to prevent joule heating.

1.2 Structured aimed Scope of Project

The project's goal is therefore to create a time-dependent software simulation of temperature as a function of position. The program will allow the user to choose the geometric surface they want to simulate in order to allow for various possible setups and use cases (different n-th dimensional polygons can be modeled via the base plate). The program will also allow the user to place as many heat sources (be it oscillating or constant in nature) on the simulated surface to see how that affects the heat map. By varying these simulation parameters we were able to model two interesting applied use cases of our software, detailed below:

1.2.1 Temperature Simulation in Water to Optimize fish breeding

Our first applied use for this program was the Fish Breeding optimization problem. In this particular scenario we added the following to our base program:

- User selected fish species: the user gets to pick which species of fish he wants to simulate for (as different fish breed at different optimal temperatures). With that in mind, the program scans the surface for areas where the temperature lies in between the species' reproductive zone. The code then plots in green the area of reproduction likelihood for the fish, allowing the user to see which configuration of heat sources maximizes future generations of fish offspring.

1.2.2 Temperature Simulation in Computer Material to Optimize Heat flow

Our second applied use for this program was to find the best setup for computers by optimizing heat flow (such that we could prevent electronic inefficiencies such as joule heating). To do so we added the following to the code base:

- User selection of time–varying, steady, oscillating modes of simulation: allows the user to chose the modes of the simulation, helps the user determine how to best setup a computer with regards to oscillating heat components of computers (such as the GPU, RAM, Power Supply, and CPU).
- User selected computer material: the user is allowed to choose a material of their choice in order to determine what material best fits the temperature flow needs of the system.

2 Theory

The software is essential modeling a time dependent 2D simulation of heat as a function of time. The physical equation that represents this behavior is given by the Transient Heat Equation:

$$\frac{\partial \mathrm{T}}{\partial \mathrm{t}} = \alpha \left(\frac{\partial^2 \mathrm{T}}{\partial \mathrm{x}^2} + \frac{\partial^2 \mathrm{T}}{\partial \mathrm{y}^2} \right) \tag{1}$$

with:

- **T** as the temperature at a position (x,y) at a specific time t.
- **t** as the time.
- $\{x, y\}$ as the spatial coordinates.
- α as the thermal diffusivity of the material, which varies depending on material composition.

Our project will therefore solve this partial differential equation to represent the evolution of heat over time.

3 Methods

3.1 Methods to create Simulation "Space"

As per eq:1 we are going to have simulate to simulate the evolution of temperature over a set of $\{x, y\}$ coordinates. To do so, we create an array of points which will represent this $\{x, y\}$ grid map. Here is the extracted snippet of the code we used (comments removed to avoid redundancy):

```
Lx, Ly = 3.0, 3.0
nx, ny = 501, 501
dx, dy = Lx/(nx-1), Ly/(ny-1)
x = np.linspace(-Lx/2, Lx/2, nx)
y = np.linspace(-Ly/2, Ly/2, ny)
X, Y = np.meshgrid(x, y)
points = np.column_stack((X.ravel(), Y.ravel()))
```

In line 1 we are defining the physical dimensions of our simulation's "space". This is an arbitrary decision as we won't simulate on all space but rather on a portion of that space. Here we choose 3 for the length L_x and L_y to create a spatial square.

In line 2 we define the resolution of our grid. We will use this resolution to crease the segments $\{dx, dy\}$ of our simulation. A higher $\{n_x, n_y\}$ will lead to a small $\{dx, dy\}$, so a more precise simulation but at the same time a more computational intensive program. We choose to use $\{n_x, n_y\}$=501 because this was the perfect balance of graphic grid and time. The choice of an odd $\{n_x, n_y\}$=501 will be explained in lines 4 and 5.

In line 3, we create the spatial step size we will use in our simulation by dividing the spatial dimensions by the resolution.

In lines and 4 and 5 we create the vector representing the coordinates along the spatial dimensions. The choice of an odd $\{n_x, n_y\}$ ensures that our vector encompasses the origin. By using the numpy mesh-grid function we use those vectors to create the matrix representation of each coordinate at every point on the grid. This gives us two arrays of dimension $\{n_x \times n_y\}$ represented by:

$$\mathbf{X} = \begin{bmatrix} x_1 & x_2 & ... & x_{n_x} \\ x_1 & x_2 & ... & x_{n_x} \\ : & : & : & : \\ x_1 & x_2 & ... & x_{n_x} \end{bmatrix} \qquad \mathbf{Y} = \begin{bmatrix} y_1 & y_1 & ... & y_1 \\ y_2 & y_2 & ... & y_2 \\ : & : & : & : \\ y_{n_y} & y_{n_y} & ... & y_{n_y} \end{bmatrix} \tag{2}$$

In line 6, we use the {**X, Y**} matrix we created in the numpy column stack function to get the 1D array containing all the points in our grid. This list has dimensions $\{n_x \cdot n_y, 2\}$ and

is represented by:

$$\text{Points} = \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \\ : & : \\ x_{n_x \cdot n_y} & y_{n_x \cdot n_y} \end{bmatrix} \tag{3}$$

With these matrices created we will be able to use them for our computation.

3.2 Methods to create Simulation Geometry

After having defined our general grid coordinates we want to create an area where we can specify our simulation; we don't want to simulate for heat propagation on a simple square but on all types of shapes with various n-sides. Because boundary conditions are crucial in solving partial differential equations, we wanted to be able to simulate for any n-side shape rather than just a square to allow for as many use cases as possible.

To achieve this we first asked the user for their input of how many sides they desire the shape to have. Then we are able to use that to create the corresponding geometric figure, and create a mask that contains the figure. This "simulation zone" will therefore be a subpart of the entire space which we created in part one. It is important to note that here we decided to set a boundary condition that points outside the "simulation zone" could be considered to be in free space, i.e. the temperature would be zero. Here is how we achieved this in our code:

```
k_sides_shape = int(input('Number of sides (e.g., 6 for a hexagon): '))
vertices = regular_polygon(k_sides_shape, polygon_radius)
vertices = np.vstack([vertices, vertices[0]])
polygon_path = Path(vertices)
mask = polygon_path.contains_points(points).reshape((ny, nx))
```

Once we run this segment return of mask of boolean values on whether that grid point is in the shape or not. To enforce boundary conditions we will simply execute a simple:

```
L[~mask] = 0
```

The L here is part of a function that will be discussed later, however, the idea here is that we are setting that points temperature at 0 because it isn't being simulated as it isn't in the shape as discussed above.

3.3 Methods to represent the Transient Heat Wave Equation

As explained by eq:1 we are solving for second derivatives of space. Unfortunately computers cannot take limits, so we have to do computational approximations. In our case we decided to use second-order central finite difference approximation. As per the central finite approximation we have:

$$\left.\frac{\partial^2 \mathrm{T}}{\partial \mathrm{x}^2}\right|_{ij} \approx \frac{\mathrm{T}_{i,j+1} - 2\mathrm{T}_{i,j} + \mathrm{T}_{i,j-1}}{dx^2} \tag{4}$$

$$\left.\frac{\partial^2 \mathrm{T}}{\partial \mathrm{y}^2}\right|_{ij} \approx \frac{\mathrm{T}_{i+1,j} - 2\mathrm{T}_{i,j} + \mathrm{T}_{i-1,j}}{dy^2} \tag{5}$$

$$\tag{6}$$

And the Laplacian can be written as:

$$\nabla^2 \mathrm{T}\big|_{ij} = \frac{\partial^2 \mathrm{T}}{\partial \mathrm{x}^2} + \frac{\partial^2 \mathrm{T}}{\partial \mathrm{y}^2} \tag{7}$$

$$\nabla^2 \mathrm{T}\big|_{ij} \approx \frac{\mathrm{T}_{i,j+1} - 2\mathrm{T}_{i,j} + \mathrm{T}_{i,j-1}}{dx^2} + \frac{\mathrm{T}_{i+1,j} - 2\mathrm{T}_{i,j} + \mathrm{T}_{i-1,j}}{dy^2} \tag{8}$$

Since we are simulating a square we can set dx = dy, and our approximation of the Laplacian gives:

$$\nabla^2 \mathrm{T}\big|_{ij} \approx \frac{\mathrm{T}_{i+1,j} + \mathrm{T}_{i,j+1} - 4\mathrm{T}_{i,j} + \mathrm{T}_{i,j-1} + \mathrm{T}_{i-1,j}}{dx^2} \tag{9}$$

Below is the extract of our code we implemented with that approximation in mind:

```
def calculate_laplacian(T_array):
    '''
    calculates the laplaciants of the array of points
    T_array - np.array: array containing all the temperatures
    '''
    L = np.zeros_like(T_array)
    L[1:-1, 1:-1] = (
        (T_array[2:, 1:-1] + T_array[:-2, 1:-1] + T_array[1:-1, 2:] + T_array
            ↪ [1:-1, :-2] - 4*T_array[1:-1, 1:-1]) / (dx**2)
    )
    L[~mask] = 0
    return L
```

In this function, we first create the an array of dimensions of the imputed array to store our Laplacian. Then we select the interior of our array with L[1:-1, 1:-1] to ensure that we don't access points outside of the grid. We then compute the $\mathrm{T}_{i,j+1}$ for each neighbor points by slicing the array. For instance the $\mathrm{T}_{i,j+1}$ is represented T_array[1:-1, 2:] line as 1:-1 selects the identical interior column and 2: shifts the column to the right like what we derived in our central approximation.

To be able to represent the time derivative we again forward finite difference. Here we have as per eq:1:

$$\frac{\partial \mathrm{T}}{\partial t} = \alpha \nabla^2 \mathrm{T} \tag{10}$$

This can be approximated with forward finite difference to be:

$$\frac{\mathrm{T}^{n+1} - \mathrm{T}^n}{\Delta \mathrm{t}} = \alpha \nabla^2 \mathrm{T} \tag{11}$$

$$\mathrm{T}^{n+1} = \alpha \nabla^2 \mathrm{T} \Delta t + \mathrm{T}^n \tag{12}$$

We easily implemented this in our code with the lines:

```
L = calculate_laplacian(T_current)
T_new = T_current + alpha * dt * L
```

This line was implemented in a for loop that runs for the amount of time we want to simulate with a Δt we choose. In most cases we chose the time elapsed to be 10s.

3.4 Methods to create heat sources

To create heat sources we will first ask the user to decide where, how strong, and what type of heat source they want to place. To improve UX experience we achieved this using matplolib and user interact. The user has to simply click on the plot and that will create the type of heat source they desire at that respective location. The user has a handy slide bar to select the temperature they desire. Here is the following code snippet:

```
def ploting_shape_and_getting_charges(path):
    '''
    gets desired points & tempetaure
    input - array: path list
    output - arrays: initial conditions
    '''
    patch = mpatches.PathPatch(path, facecolor='none', lw=2)

    fig, ax = plt.subplots(figsize=(8, 8))

    ax.add_patch(patch)
    ax.set_xlim(-Lx, Lx)
    ax.set_ylim(-Ly, Ly)

    ax.set_aspect('equal')

    plt.subplots_adjust(bottom=0.4)

    static_slider_ax = plt.axes([0.3, 0.30, 0.4, 0.05])
    static_temp_slider = Slider(
        ax=static_slider_ax,
        label='Static Temperature',
        valmin=-100,
        valmax=100,
        valinit=0
    )

    dynamic_slider_ax = plt.axes([0.3, 0.20, 0.4, 0.05])
    dynamic_temp_slider = Slider(
        ax=dynamic_slider_ax,
        label='Oscillating Temperature',
        valmin=-100,
        valmax=100,
        valinit=0
    )

    osci_slider_ax = plt.axes([0.3, 0.10, 0.4, 0.05])
    osci_freq_slider = Slider(
        ax=osci_slider_ax,
        label='Oscillating Frequency',
        valmin=0,
        valmax=2,
        valinit=0
    )

    button_ax = plt.axes([0.4, 0.02, 0.2, 0.05]) # [left, bottom, width,
        ↪ height]
    submit_button = Button(button_ax, 'Submit')

    rows_static = []
    cols_static = []
    rows_osci = []
    cols_osci = []

    static_temp = [0]
    dynamic_temp = [0]
    osci_freq = [0]
    is_submitted = [False]


    def on_slide_stat_temp(val):
        static_temp[0] = val
        fig.canvas.draw_idle()

    def on_slide_osci_temp(val):
        dynamic_temp[0] = val
        fig.canvas.draw_idle()
```

```
    def on_slide_freq(val):
        osci_freq[0] = val
        fig.canvas.draw_idle()

    def onclick(event):
            if event.inaxes != ax:
                return

            x_event, y_event = event.xdata, event.ydata

            ix = np.argmin(np.abs(x - x_event))
            iy = np.argmin(np.abs(y - y_event))

            snapped_ix = (ix // sample_rate) * sample_rate
            snapped_iy = (iy // sample_rate) * sample_rate

            x_snapped_coord = x[snapped_ix]
            y_snapped_coord = y[snapped_iy]

            if event.button == 1:
                rows_static.append(snapped_ix)
                cols_static.append(snapped_iy)
                ax.plot(x_snapped_coord, y_snapped_coord, 'ro', label='Static
                    ↪  Source')

            elif event.button == 3:
                rows_osci.append(snapped_ix)
                cols_osci.append(snapped_iy)
                ax.plot(x_snapped_coord, y_snapped_coord, 'bo', label='
                    ↪ Oscillating Source')

            plt.draw()

    def on_submit(event):
        is_submitted[0] = True
        plt.close(fig)

    cid = fig.canvas.mpl_connect('button_press_event', onclick)
    static_temp_slider.on_changed(on_slide_stat_temp)
    dynamic_temp_slider.on_changed(on_slide_osci_temp)
    osci_freq_slider.on_changed(on_slide_freq)
    submit_button.on_clicked(on_submit)

    plt.show()

    return cols_static, rows_static, cols_osci, rows_osci, static_temp[0],
        ↪ dynamic_temp[0], osci_freq[0], is_submitted[0]
```

To explain this code perhaps a screenshot is more relevant, so here below is what the user input looks like.

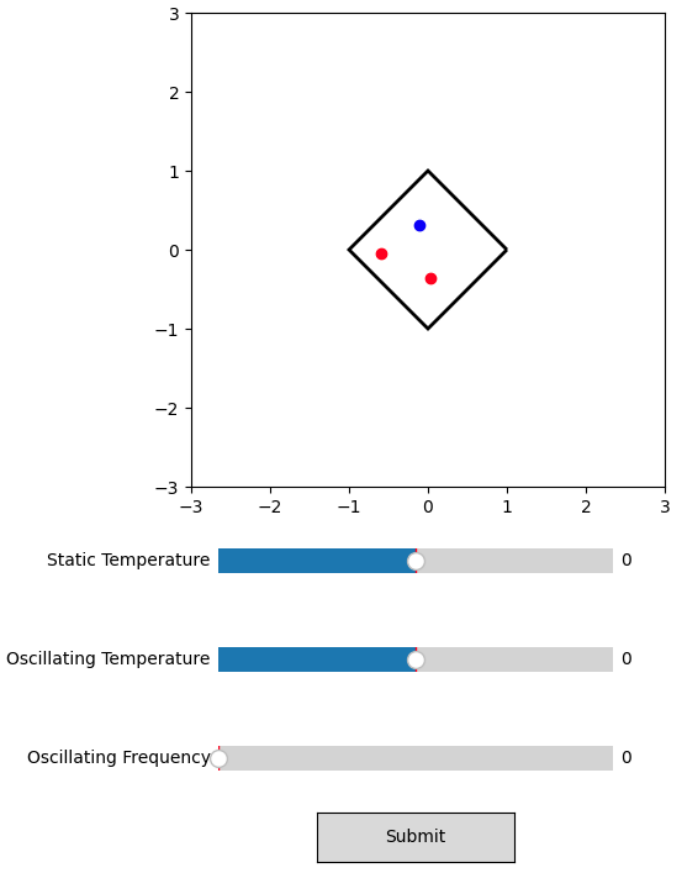

Figure 2: Example User Placement Of Heat Sources (Red = Static, Blue = Oscillating)

The red points are static (constant) heat sources, and blue ones are oscillating. It is important to note that we are able to get a list of all the coordinates with heat sources and their properties, and then use our finite differential approximation to force that temperature in the code as per the user's request. See below for the implementation of that code:

```
L = calculate_laplacian(T_current)
T_new = T_current + alpha * dt * L
current_source_temp = tempetarure_dynamic * np.sin(omega * total_sim_time) #
    ↪ Asin(wt)
if len(cols_static) > 0:
    T_new[cols_static,rows_static] = tempature_static
if len(cols_dynamic) > 0:
    T_new[cols_dynamic,rows_dynamic] = current_source_temp
T_new = apply_boundary_conditions(T_new)
```

This code enforces that the temperature at the selected grid points is the user's desired heat source. Because the Laplacian computes neighbors of the points, this will ensure that the heat sources inputted do in fact have an impact on the temperature matrix we compute.

3.5 Methods to find area of Breed-ability

As per the theory for the heat wave equation, our code computes the temperature matrix for each Δt of the runtime. We will use this to achieve a very straightforward check to see if the temperature is within a range. In our code we do this using numpy with the following line:

```
T_bound_frame = np.where(
```

```
            (T_plot_frame >= fish_lower_bound_temperature) & (T_plot_frame <=
                ↪ fish_upper_bound_temperature),
            graphical_highlighting_for_survival_bound, np.nan
        )
```

Here, we are simply creating a matrix that contains the points that are within the bound temperature of the breeding temperature of the fish specimen. We then plot this matrix with numpy on the grid with a green spot to indicate to the user that this "zone" is within the range of the specimen.

To improve UX we simply ask when we start the code what species the user wants to simulate for and then run this string into a dictionary to get the bounds. Here is the code we use to do so:

```
fish_temperature_information = {
    "dorade": [2,8],
    "bar": [1,5],
    "espadon": [14,15],
    "shark": [1,12.5],
    "gold fish": [5,15],
    "lava fish": [40,100],
    "test": [10,100]
}

print('Please now select the fish to simulate. Here is our selection of
    ↪ possible fishes today:')
    print('- dorade - bar - espadon - shark - small fish - lava fish')
    fish_type = str(input('input the fish you want to simulate: '))
    fish_list_bound_information = fish_temperature_information.get(fish_type)
    fish_lower_bound_temperature = fish_list_bound_information[0]
    fish_upper_bound_temperature = fish_list_bound_information[1]
```

3.6 Methods to find the Modes of simulation

In our second applied case we want to find the modes of the simulation, so the steady and oscillating modes. To do so, we will use a neat property of SVD. In particular as per the definition of the SVD decomposition, it will transform a matrix A into three new matrices that inherit separate properties. By definition:

$$\mathbf{A}_{N_{space} \times N_{time}} = \mathbf{U}_{N_{space} \times N_{space}} \mathbf{S}_{N_{space} \times N_{time}} \mathbf{V}^{\mathrm{T}}_{N_{time} \times N_{time}} \tag{13}$$

In this example the matrix $\mathbf{U}$ will inherit the spatial modes, $\mathbf{S}$ the magnitudes, and $\mathbf{V}^{\mathrm{T}}$ the temporal modes. The spatial modes represent the spatial patterns of the matrix. The magnitudes represent the significance of each spatial mode. The temporal modes represent the amplitude modulation associated with each spatial mode. By implementing this SVD decomposition we are able to get key information from our simulation, notably the principal spatial mode represented by the $\mathbf{U}$ matrix. This gives us an accurate description of the fluctuation directly caused by our heat sources. In other words, by plotting this mode we get information not only on the average but also the fluctuation from the average.

Because we are interested in the deviation from the average, we will first have to take the average of our data to center a new matrix. By taking SVD on this newly centered matrix, we will be able to effectively capture the spatial modes of our data. Taking the average of our temperature matrix also allows us to get information on the steady state behavior of our simulation as we can approximate the steady state to be the average fluctuation. Below is how we implemented this method in our code (comments removed to avoid redundancy):

```
Data_Matrix = np.array([T.ravel() for T in T_frames]).T
average_temperature = np.mean(Data_Matrix, axis=1)
temperature_approximated_steady_centered = Data_Matrix - average_temperature
    ↪ [:, np.newaxis]
U, s, Vh = np.linalg.svd(temperature_approximated_steady_centered,
    ↪ full_matrices=False)

T_steady_2D = average_temperature.reshape((ny, nx))
T_steady_plot = T_steady_2D[::plot_subsample_rate, ::plot_subsample_rate]
T_oscillation_mode_2D = U[:, 0].reshape((ny, nx))
T_oscillation_plot = (T_oscillation_mode_2D * s[0])[::plot_subsample_rate, ::
    ↪ plot_subsample_rate]
```

In lines 1 we are first creating a matrix of dimensions $\{N_{time}, N_{space}\}$ by using the numpy ravel function. The transpose is here to make the new matrix have dimensions $\{N_{space}, N_{time}\}$ to comply with SVD formatting. Here is an example of what that matrix would look like:

$$\text{Data Matrix} = \begin{bmatrix} \mathrm{T}^{t_1}_{x_1,y_1} & \mathrm{T}^{t_2}_{x_1,y_1} & ..\mathrm{T}^{t_N}_{x_1,y_1} \\ \mathrm{T}^{t_1}_{x_2,y_2} & \mathrm{T}^{t_2}_{x_2,y_2} & ..\mathrm{T}^{t_N}_{x_2,y_2} \\ \vdots & \vdots & \vdots \\ \mathrm{T}^{t_1}_{x_N,y_N} & \mathrm{T}^{t_2}_{x_N,y_N} & ..\mathrm{T}^{t_N}_{x_N,y_N} \end{bmatrix} \tag{14}$$

In line 2, we are then able to take the time dimension axis and average our matrix to find what we approximate to be the steady state representation. We then reshape that into a 2D matrix for later plotting, in lines 6 and 7.

In line 4, after having correctly set up our centered matrix, we are able to take the SVD using numpy. We then reshape that vector into a 2D matrix for later plotting. As we have reshaped the temperature array, this **U** matrix will represent the primary fluctuations due to our heat sources.

3.7 Practice Example

After explaining how our code works, let us demonstrate it visually using a trivial example of a simple constant heat source in the center of a circle with an arbitrary α.

The code will first ask the user for shape, here we answer 20 to get an approximation of a circle.

```
Number of sides (e.g., 6 for a hexagon): 20
```

we then select a single static temperature point of arbitrary temperature (here 100 degrees) at the center of our circle which we show here:

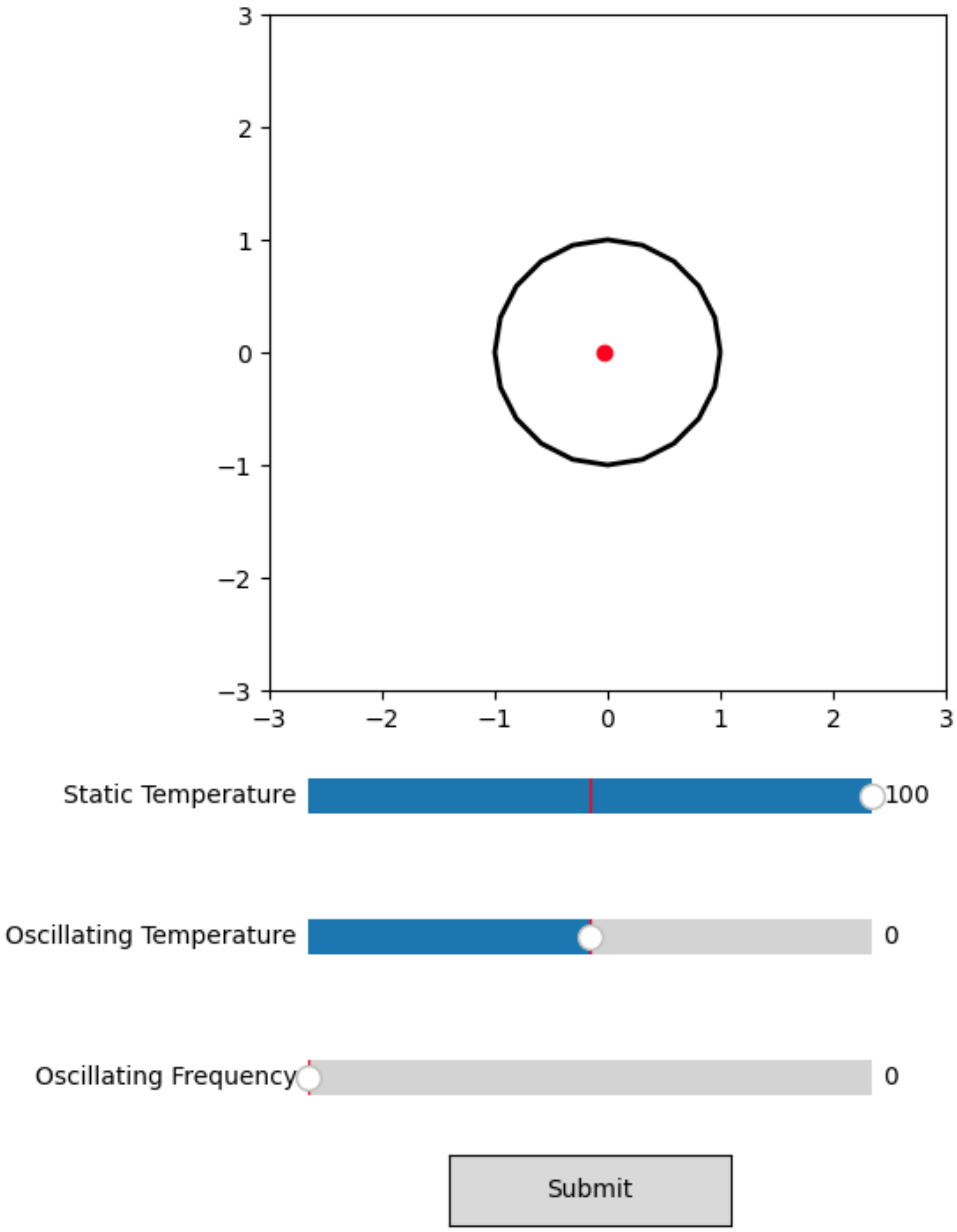

Figure 3: example of the user interface for a static point of temperature

We then have our simulation which is a time dependent simulation. Here is a screenshot of the final evolution. However you can also find a link to a video recording containing that simulation: `https://youtube.com/shorts/2rXXrk2Xu6o?feature=share`.

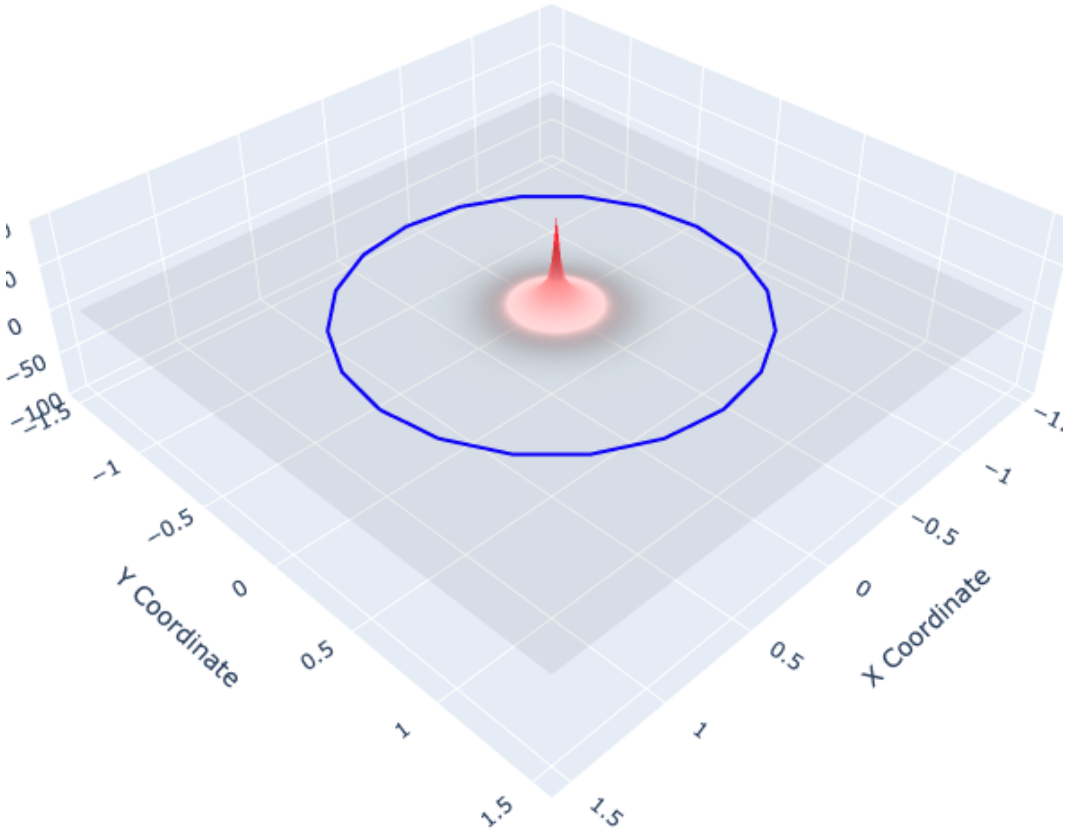

Figure 4: simulation result with an (n = 20)-th dimensional polygon and 1 static heat source (100 degrees).

4 Results

In our project we will have two main results. Firstly the results of the fish simulation and then the results of our computer architecture experiment.

4.1 Results on the fish simulation experiment

Before getting into further analysis let us first establish some base conditions for this problem:

- There are 4 goldfish in a pond.
- We have 4 static heat sources (that can reach up to 20 degrees each) in a effort to keep them alive. For the purpose of this scenario we've attempted to set up the heat sources at up to 10 degrees on each corner of the pond.

- Economically, it would not make sense to buy more heat sources, so 4 is the maximum number of heat sources available.
- We are trying to optimize the heating of the pond without having to increase the power supply to our heaters and without purchasing any more heat sources (how can we maximize efficiency and economically make the best decision to heat up the pond to an idealized temperature so that these goldfish can reproduce?)
- The pond is very shallow.
- The pond can be approximated to be 1×1 meters in dimension.

Our problem is the following. We want to find the best placement of the 4 static heat sources to maximize the correct thermal area for the goldfish to reproduce. Through various simulated setups we have identified that a dense pattern with constant temperature at around slightly higher than the average reproductive temperature of the goldfish, leading to the highest area of survivability (see figure:6).

The original attempt (see figure:8) did not work because the grid points were too far apart, which led to a very small heat propagation in the pond. What the simulation crucially demonstrated to us was that the heat sources needed to be closer to each other in order to maximize the thermal distribution.

Here is an example of the highest yield we experienced. The screenshots are the screenshots at the end of the simulation. For a better idea of propagation, please see the video linked here: `https://www.youtube.com/watch?v=9yP1qq0rQc0&feature=youtu.be`

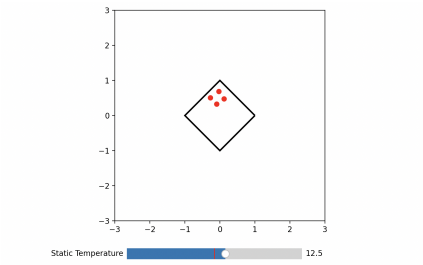

Figure 5: User Interface for Optimal Thermal Distribution Modeled by 4 Static Heat Sources

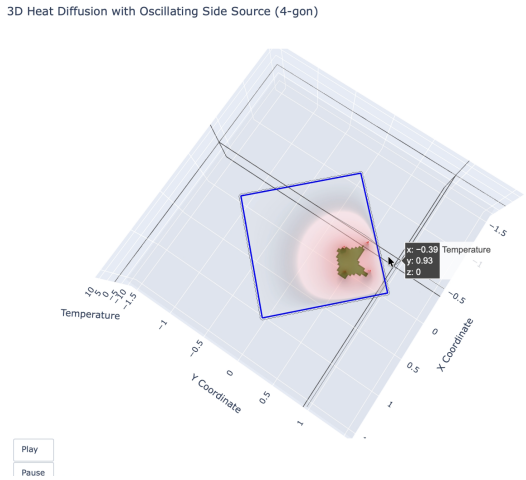

Figure 6: Simulated Optimal Thermal Distribution (4 Static Sources Setup) Result

```
Please now select the fish to simulate. Here is our selection of possible
    ↪ fishes today:
- dorade - bar - espadon - shark - gold fish - lava fish
input the fish you want to simulate: gold fish
```

Here is what the original setup (four static sources spaced as far apart as possible) yielded. We can see that no green zones occurred, meaning a lack of goldfish reproduction was to be expected. For a better idea of the thermal propagation please see the video linked here: `https://youtu.be/zFH4a8tk_UI`

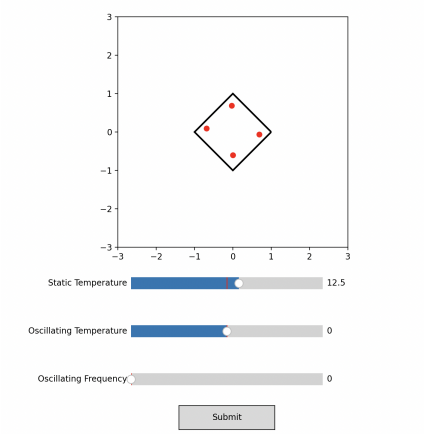

Figure 7: Original Heating Setup With 4 Static Sources

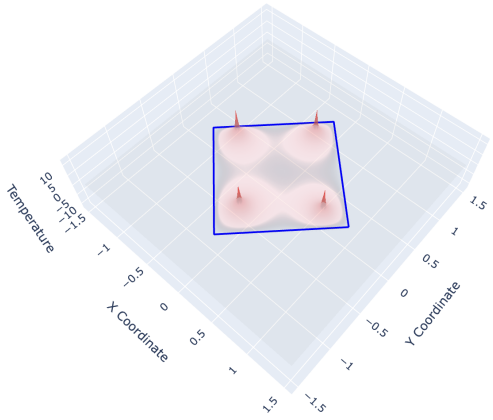

Figure 8: Simulated Thermal Distribution Results (4 static sources distanced as far apart as possible

```
Please now select the fish to simulate. Here is our selection of possible
    ↪ fishes today:
- dorade - bar - espadon - shark - gold fish - lava fish
input the fish you want to simulate: gold fish
```

4.2 Results on the computer architecture simulation experiment

Let's establish the key components of our problem:

- Each computer has 3 main components that sit on a motherboard: GPU, CPU, RAM, Power. We want to simulate this architecture on different motherboard materials to find which material best suits this architecture.
- Because the GPU is inherently dependent on computer load we can simulate the GPU as being an oscillating power source (of around 70 degrees.) CPU, RAM, and the Power Source can be assumed to be constant (static sources of heat).
- We will simulate for two common materials used in computers: copper and aluminum. We approximate that these materials will dominate in the alpha (thermal diffusivity) of the material. We approximate in our models that each simulation behaves exclusively with the chosen material.

In this problem we want to simulate different materials in our PC and see which one allows for the best heat behavior of the PC. In this case, best means colder (minimized thermal distribution). We also want to try and see what placement of the GPU and CPU on the motherboard best helps the heat flow on the computer.

4.2.1 Results on the Copper PC

`https://youtu.be/oBfFfIsTDGY`

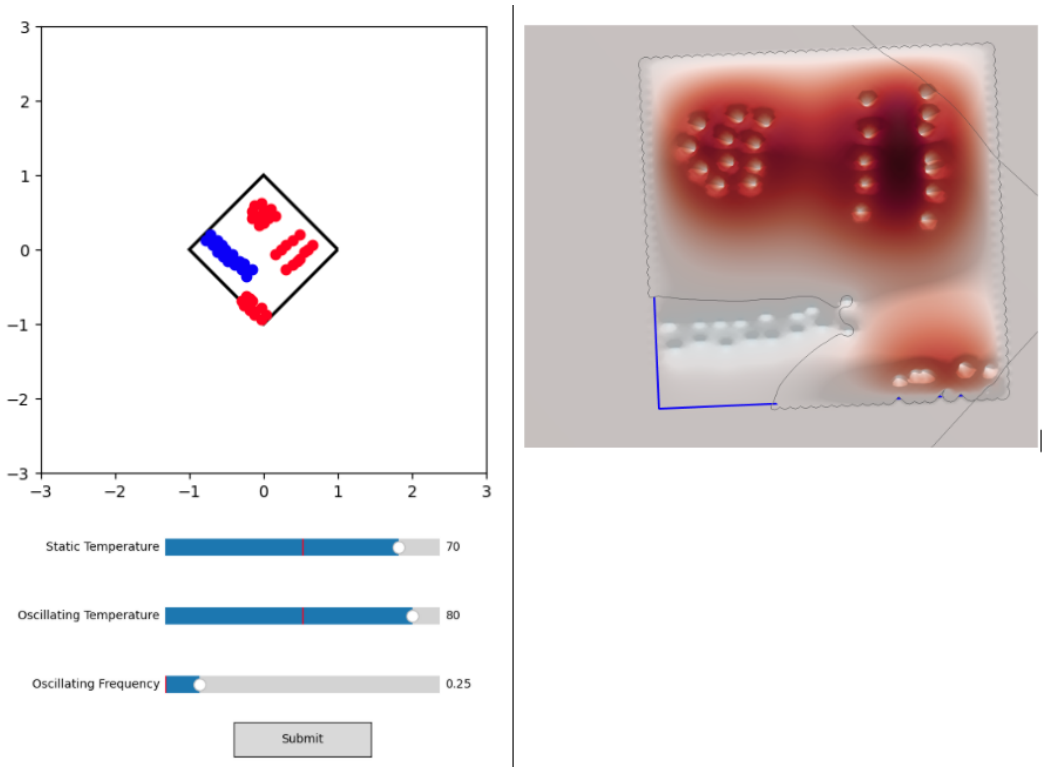

Figure 9: Modeling of Copper PC Components ($\alpha = 1.12 \cdot 10^{-4}$) Simulation Results

```
Please now select the PC material. Here is our selection:
- copper - alimunium - FR-4 -
input the material you want to simulate: copper
```

4.2.2 Modeling of Aluminum PC Components ($\alpha = 80 \cdot 10^{-6}$) Simluation Results

`https://youtu.be/7wCciacZRVw`

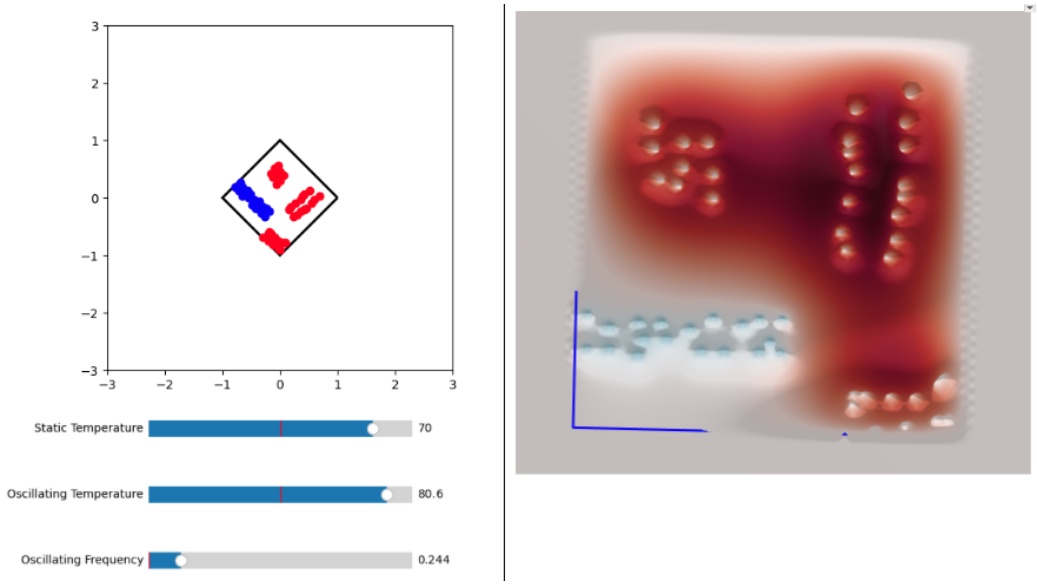

Figure 10: Aluminium PC Simulation ($\alpha = 80 \cdot 10^{-6}$) Results

```
Please now select the PC material. Here is our selection:
- copper - alimunium - FR-4 -
input the material you want to simulate: alimunium
```

4.2.3 Results on different GPU and CPU placement (copper material)

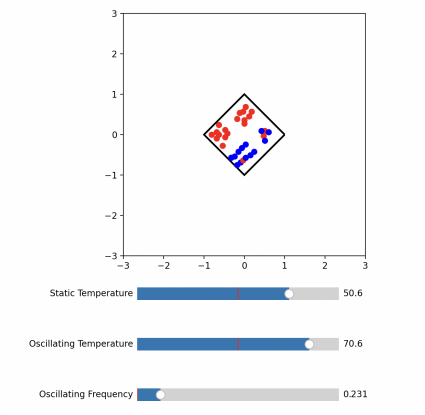

Figure 11: Modeling Different CPU and GPU Component Placements in a PC

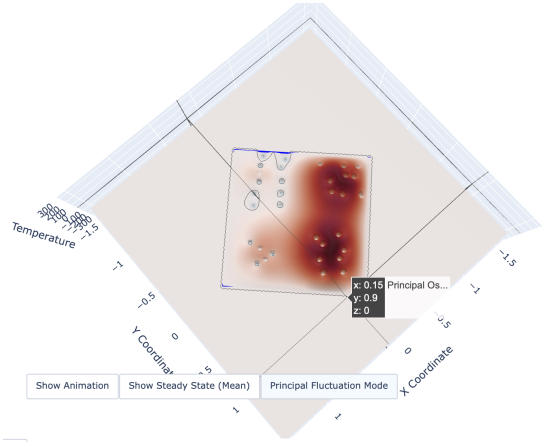

Figure 12: Thermal distributions for differently spaced placements of CPU and GPU components.

4.2.4 Analysis of Results for best computer material

As per our results we can observe that the copper response to the heat fluctuations spreads the heat out more evenly than the aluminum one. The aluminum has a greater thermal response (hence the darker red than that in the copper simulation), meaning it will produce more heat, and do so in concentrated areas. Indeed, as seen in figures 9 and 10, the heat response map for copper is more distributed than the one for aluminum. Heat spreading is

desired in computer heat flow, as it will allow for the cooling of computer components as much as possible. Thus, we can confidently conclude that copper is the more appropriate material for PCs (which also happens to align with the fact that copper is a greater conductor of heat than aluminum, meaning it is less likely to contribute to thermal inefficiencies in computer systems).

4.2.5 Analysis of Results for best location for GPU and CPU

As per our results in fig:12 we see that another way of positioning the CPU and GPU actually leads to an undesirable response to temperature. If we don't place both main heat sources CPU and GPU as far apart as we can, hotter zones in the PC will appear, leading to components losing the ability to cool passively. We can thus conclude from our data that the current locations of CPU and GPU already optimize heat flow (when they are spread as far apart as possible).

5 References

1. Fish reproductive success occurs within narrow ΔT (3.5):
   Pankhurst, Ned Munday, Philip. (2011). Effects of climate change on fish reproduction and early life history stages. Marine and Freshwater Research. 62. 1015-1026. 10.1071/MF10269.

2. SVD Interpretation (3.6):
   Su, Chenyao, Yong Hu, Yiwang Ma, and Jiuling Yang. 2025. "Simulation Study and Proper Orthogonal Decomposition Analysis of Buoyant Flame Dynamics and Heat Transfer of Wind-Aided Fires Spreading on Sloped Terrain" Fire 8, no. 4: 139. https://doi.org/10.3390/fire8040139